

UD7: Monitorización y optimización. Auditoría

Índice

Introducción.....	2
Registros de auditoría.....	2
Auditoría básica.....	3
AUD\$.....	3
Control de los registros de auditoría.....	4
Auditorías de inicio.....	4
Auditorías de acción o privilegio.....	5
Auditorías de Objeto.....	6
Auditoría extendida.....	6
Consultar la auditoría.....	7
Vista sobre los privilegios y sentencias.....	7
DBA_PRIV_AUDIT_OPTS.....	7
DBA_STMT_AUDIT_OPTS.....	7
Vista sobre los objetos.....	7
DBA_OBJ_AUDIT_OPTS.....	7
Noaudit.....	8
de sesión.....	8
de objeto.....	8
Auditoría Fina.....	8
Flashback.....	10
Flashback Version Query (FVQ).....	10
Flashback Transaction Query (FTQ).....	12

Introducción

La monitorización es el proceso que se lleva a cabo para evaluar cómo los usuarios usan la base de datos. La monitorización y el registro de los eventos de usuario que se desean evaluar se denomina auditoría. Las auditorías en bases de datos se pueden desarrollar en diferentes niveles de profundidad atendiendo a las actividades que se proponen estudiar y a los planes de acción que se desarrollan con la base de datos para su optimización y la resolución de problemas.

Registros de auditoría

Los sistemas gestores de bases de datos permiten configurar las actividades que se registran en los registros de auditoría. Un registro de auditoría es el almacenaje de cualquier acción de interés para luego usarlo en los procesos de análisis, tomas de decisiones y control. Oracle es muy robusto en este tipo de operaciones y se puede registrar casi todo lo que un usuario hace con la base de datos. Lo que se requiere es indicar que operación se necesita registrar, el usuario que la hizo, la fecha y la hora, y el objeto que interviene en la operación. Estos registros se almacenan en el diccionario de datos o en ficheros propios del sistema operativo.

La primera técnica se conoce como database audit trail o DBAT y la segunda operating system audit trail u OSAT. Las acciones que las cuentas de administración llevan a cabo sobre la base de datos pueden ser registradas en el banco de registros de auditoría de la base de datos. Para que esto ocurra, se debe poner a true el parámetro `AUDIT_SYS_OPERATION`. Esto se hace cuando los responsables de las auditorías son diferentes a los administradores de la base de datos. Este sería el primer tipo de registro de auditoría.

Otros registros de auditoría son aquellos que almacenan algunos valores de algunas acciones, es decir, su nivel de auditoría es básico. Estos registros, denominados **registros de auditoría estándares**, recaen sobre la ejecución de algunos comandos, las operaciones sobre algunas tablas, los procesos de autenticación y el control de algunos privilegios. Estos registros se escriben en la tabla `AUD$`, accesibles con la vista `DBA_AUDIT_TRAIL`, o en unos ficheros que se pueden definir.

Otro tipo de auditoría, denominada **auditoría fina** (FGA, Fine-Grained Auditing), permite guardar información relativa a las operaciones que se realizan sobre las filas o columnas, lo que la convierte en una auditoría más profunda.

En la vista `DBA_COMMON_AUDIT_TRAIL` se almacenan los registros estándares y los registros finos.

Además de este tipo de mecanismos, las auditorías se pueden almacenar usando los disparadores o **triggers** en bloques PL/SQL.

Auditoría básica

La auditoria estándar o básica se activa usando el parámetro **AUDIT_TRAIL**. Cuando este parámetro vale DB los registros de auditoria se almacenan en la tabla **AUD\$**. Estos registros se pueden redirigir a un fichero en lugar de hacerlo a AUD\$, sobre todo cuando pretende auditar al propio administrador, ya que los registros en AUD\$ son 'modificables' por este. Para este propósito, el parámetro AUDIT_TRAIL se pone a OS y se indica el fichero o ficheros de destino con el parámetro **AUDIT_FILE_DEST**. Por defecto \$ORACLE_HOME/rdbms/audit. El formato que Oracle usa para esta opción es un fichero plano.

En el caso en que se desee almacenar la auditoria en ficheros con el formato **XML**, el parámetro AUDIT_TRAIL valdrá XML. Por ejemplo podemos hacer

ALTER SYSTEM SET AUDIT_TRAIL=DB SCOPE=SPFILE;

Parar y levantar a base de datos.

AUD\$

El diccionario de datos de cada base de datos tiene una tabla denominada SYS.AUD\$, comúnmente denominada registro de auditoría de la base de datos , que está diseñada para almacenar entradas que auditan declaraciones de base de datos, privilegios u objetos de esquema.

Opcionalmente, puede optar por almacenar la información de auditoría de la base de datos en un archivo del sistema operativo. La ventaja de usar la tabla es que puedes ver partes seleccionadas del registro de auditoría con las vistas de registro de auditoría predefinidas del diccionario de datos.

El registro de auditoría de la base de datos, almacenado en la tabla SYS.AUD\$, contiene distintos tipos de información, según los eventos auditados y las opciones de auditoría establecidas. La siguiente información siempre se incluye en el registro de auditoría:

Nombre de usuario (**userid** o **current_user**)

Identificador de sesión (**sessionid**)

Nombre del objeto de esquema al que se accedió

Operación realizada o intentada (**action#** Ver AUDIT_ACTIONS)

Código de finalización de la operación (**returncode** cero si bien o error ORA-)

El registro de auditoría no almacena información sobre los valores de los datos que puedan estar involucrados en la declaración auditada. Por ejemplo, los valores de datos antiguos y nuevos de las filas actualizadas no se almacenan cuando se audita un UPDATE. Sin embargo, este tipo especializado de auditoría se puede realizar utilizando métodos de auditoría de granularidad fina.

Por ejemplo podemos hacer este select:

select userhost,sessionid, action#,current_user,returncode from aud\$;

Control de los registros de auditoría

Lo siguiente que se debe hacer es indicar aquellos elementos individuales que se quieren auditar. Se podrá auditar un conjunto de privilegios y un conjunto de comandos SQL. Para indicar el comando o privilegio que auditar se usa el comando AUDIT seguido del elemento.

**AUDIT {Consulta | objeto | NETWORK} [BY {SESSION | ACCESS}]
[WHENEVER [NOT SUCCESSFUL];**

La opción BY SESSION hace referencia al registro de una violación de una auditoria por cada sesión. Las sucesivas violaciones no se registran, ya que se trata de la misma violación. Sin embargo, la opción BY ACCESS provoca un registro de auditoria cada vez que la violación se da, aunque se haya dado con anterioridad en la misma sesión.

Se debe tener en cuenta que el proceso de monitorización consume muchos recursos, ya que el motor del SGBD tiene que estar registrando todo aquello que se ha configurado para monitorizar. Así, la opción BY ACCESS solo se debe usar para aquellos objetos y operaciones que sean altamente necesarios.

Existen varios tipos de auditorias que se pueden clasificar de la siguiente manera

Auditorías de inicio: hacen referencia a las conexiones con la base de datos.

Auditorías de acción o privilegio: hacen referencia a cualquier operación create, alter o drop que afecte a un objeto de la base de datos. Pueden agruparse para simplificar la configuración de la auditoria.

Auditorías de objeto: hacen referencia a cualquier operación select, insert, update y delete

Auditorías de inicio

Estas auditorias se registran en cada conexión a la base de datos. Se usa **AUDIT SESSION**; para comenzar la auditoria. Con esta opción se registran tanto las conexiones correctas como las fallidas. Si se quisieran auditar solo las fallidas se usa el comando siguiente: **AUDIT SESSION WHENEVER NOT SUCCESSFUL;**

Para auditar solo las correctas se usaría la opción SUCCESSFUL, tal como se indica en el ejemplo siguiente: **AUDIT SESSION WHENEVER SUCCESSFUL;**

AUDIT SESSION BY usuario, usuario2,...

auditaríamos sólo ese usuario

Podemos mirar directamente la tabla AUD\$ o la vista **DBA_AUDIT_SESSION**

PRÁCTICA

AUDITORÍA DE SESIÓN

Habilita la auditoría y consulta la tabla AUD\$ y la vista DBA_AUDIT_SESSION y DBA_AUDIT_TRAIL
Mira en la vista V\$session y mata la sesión del usuario con
SQL> ALTER SYSTEM KILL SESSION 'sid,serial#' IMMEDIATE;

Auditorías de acción o privilegio

Si queremos auditar la creación, borrado y alteración de tablas de un usuario concreto ponemos la cláusula BY o podemos omitirla

AUDIT TABLE BY usuario

Cuando interesa auditar los tres tipos de operaciones, no es necesario indicarla. De este modo, el siguiente ejemplo audita las operaciones CREATE, ALTER y DROP de usuarios, es decir, cualquier operación asociada con los usuarios:

AUDIT USER;

Podemos también auditar sentencias SQL como por ejemplo:

ALTER TABLE	ALTER TABLE
DELETE TABLE	DELETE FROM <i>table, view</i>
EXECUTE PROCEDURE	CALL Execution of any procedure or function or access to any variable, library, or cursor inside a package
GRANT TABLE	GRANT privilege ON Table, view, materialized view REVOKE privilege ON table, view, materialized view
INSERT TABLE	INSERT INTO table, view
LOCK TABLE	LOCK TABLE table, view
SELECT TABLE	SELECT FROM table, view, materialized view
UPDATE TABLE	UPDATE table, view

Auditorías de Objeto

Cuando las auditorías son sobre operaciones DML en vez DDL, se usa la opción ON para indicar el objeto en cuestión. De este modo, la opción ON se usa en las operaciones insert, update, delete y select.

siguiente ejemplo audita las operaciones de inserción y actualización en la tabla profesor

```
AUDIT INSERT UPDATE ON PROFESOR BY ACCESS;
```

Al igual que en el caso anterior, se puede usar la opción BY para indicar el usuario que monitorizar. Cuando se quiere monitorizar cualquier operación sobre un objeto se usa la opción ALL. En el siguiente ejemplo, con el que se quiere auditar todas las operaciones sobre la tabla Matricula, se usa esta opción:

```
AUDIT ALL ON MATRICULA BY SESSION;
```

Auditoría extendida

Si el parámetro es AUDIT_TRAIL es db,extended me permite conocer exactamente el texto SQL que se tecleó que aparecerá en SQL_TEXT

PRÁCTICA

AUDITORIA EXTENDIDA

Supón este escenario: Una aplicación está funcionando mal, se ejecuta con el usuario de la base de datos *tienda*.

No sabemos que sentencias están fallando, así que vamos a auditarlas

Habilita la auditoría para los fallos

```
AUDIT SELECT TABLE, INSERT TABLE, DELETE TABLE, EXECUTE PROCEDURE  
  BY ACCESS WHENEVER NOT SUCCESSFUL;
```

Con el usuario tienda crea una tabla e inserta varios valores, ejecuta sentencias con errores y mira la vista dba_audit_trail (sobretudo el campo SQL_TEXT)

Consultar la auditoría

Para ver la auditoría actual podemos consultar dos vistas:

Vista sobre los privilegios y sentencias

DBA_PRIV_AUDIT_OPTS

DBA_STMT_AUDIT_OPTS

Vista sobre los objetos

DBA_OBJ_AUDIT_OPTS

Nos aparecerán las opciones, para cada una dos valores separados por una barra, el valor de la izquierda indica si se audita para los éxitos y el de la derecha para los fracasos. Los valores pueden ser:

– (no se audita)

A (se audita)

DBA_AUDIT_TRAIL					
OS_USERNAME	VARCHAR2(255)	AUDIT_OPTION	VARCHAR2(40)	SESSION_CPU	NUMBER
USERNAME	VARCHAR2(128)	SES_ACTIONS	VARCHAR2(19)	EXTENDED_TIMESTAMP	TIMESTAMP(6)
USERHOST	VARCHAR2(128)	LOGOFF_TIME	DATE	PROXY_SESSIONID	NUMBER
TERMINAL	VARCHAR2(255)	LOGOFF_LREAD	NUMBER	GLOBAL_UID	VARCHAR2(32)
TIMESTAMP	DATE	LOGOFF_PREAD	NUMBER	INSTANCE_NUMBER	NUMBER
OWNER	VARCHAR2(128)	LOGOFF_LWRITE	NUMBER	OS_PROCESS	VARCHAR2(16)
OBI_NAME	VARCHAR2(128)	LOGOFF_DLOCK	VARCHAR2(40)	TRANSACTIONID	RAW(8)
ACTION	NUMBER	COMMENT_TEXT	VARCHAR2(4000)	SCN	NUMBER
ACTION_NAME	VARCHAR2(28)	SESSIONID	NUMBER	SQL_BIND	NVARCHAR2(2000)
NEW_OWNER	VARCHAR2(128)	ENTRYID	NUMBER	SQL_TEXT	NVARCHAR2(2000)
NEW_NAME	VARCHAR2(128)	STATEMENTID	NUMBER	OBI_EDITION_NAME	VARCHAR2(128)
OBI_PRIVILEGE	VARCHAR2(32)	RETURNCODE	NUMBER	DBID	NUMBER
SYS_PRIVILEGE	VARCHAR2(40)	PRIV_USED	VARCHAR2(40)	RLS_INFO	CLOB
ADMIN_OPTION	VARCHAR2(1)	CLIENT_ID	VARCHAR2(128)	CURRENT_USER	VARCHAR2(128)
GRANTEE	VARCHAR2(128)	ECONTEXT_ID	VARCHAR2(64)		

Podemos ver los privilegios y sentencias que son auditable en las vistas [system_privilege_map](#) y [stmt_audit_option_map](#)

Noaudit

Para dejar de auditar usamos la sentencia noaudit

de sesión

NOAUDIT session;

NOAUDIT session BY usuario;

NOAUDIT DELETE ANY TABLE;

NOAUDIT SELECT TABLE, INSERT TABLE, DELETE TABLE, EXECUTE PROCEDURE;

NOAUDIT ALL;

de objeto

NOAUDIT DELETE ON emp;

NOAUDIT SELECT, INSERT, DELETE ON scott.dept;

NOAUDIT ALL ON scott.emp;

PRÁCTICA

DESHABILITA LA AUDITORÍA

Consulta la vista DBA_PRIV_AUDIT_OPTS y deshabilita una a una todas las auditorías que tengas.

Consutla la vista DBA_OBJ_AUDIT_OPTS y deshabilita una a una las auditorías que tengas sobre los objetos de los usuarios.

Pon el parámetro AUDIT_TRAIL a none y mira que no sigue auditando.

Auditoría Fina

Las auditorías finas son controles sobre algunas columnas en particular de una tabla de interés. Para configurar este tipo de registros se usa el paquete **DBMS_FGA**. Dentro de este paquete existe el procedimiento **ADD_POLICY** que permite definir las políticas para una auditoría en concreto.

Para que un usuario pueda crear una auditoría de este tipo, debe disponer del rol AUDIT_ADMIN. El procedimiento ADD_POLICY tiene la siguiente definición:

DBMS_FGA.ADD_POLICY(

object_schema	IN VARCHAR2 DEFAULT NULL,
object_name	IN VARCHAR2,
policy_name	IN VARCHAR2,
audit_condition	IN VARCHAR2 DEFAULT NULL,
audit_column	IN VARCHAR2 DEFAULT NULL,
handler_schema	IN VARCHAR2 DEFAULT NULL,
handler_module	IN VARCHAR2 DEFAULT NULL,
enable	IN BOOLEAN DEFAULT TRUE,
statement_types	IN VARCHAR2 DEFAULT SELECT,
audit_trail	IN BINARY_INTEGER DEFAULT NULL,
audit_column_opts	IN BINARY_INTEGER DEFAULT ANY_COLUMNS,
policy_owner	IN VARCHAR2 DEFAULT NULL);

Ejemplo de uso:

```
exec dbms_fga.add_policy (  
object_name=>'REGIONS',  
object_schema=>'HR',  
policy_name=>'proba',  
audit_condition=>'REGION_ID>5',  
audit_column=>'REGION_ID',  
statement_types=>'SELECT,UPDATE');
```

Para deshabilitar la auditoria se usa el procedimiento **DISABLE_POLICY** **DBMS_FGA.DISABLE_POLICY(object_schema IN VARCHAR2, object_name IN VARCHAR2, policy_name IN VARCHAR2);** con los argumentos esquema, objeto y nombre de la politica, para habilitarla **ENABLE_POLICY** Para eliminar una politica de auditoria se usa el procedimiento **DROP_POLICY** con los mismos argumentos que los dos anteriores.

La vista que se utiliza para obtener información sobre las auditorias estándares y las auditorias finas es **DBA_COMMON_AUDIT_TRAIL**. Esta vista tiene muchas columnas de interés. Un ejemplo de uso:

```
select current_user,statement_type,object_name,sql_text from dba_common_audit_trail;
```

PRÁCTICA

FGA

Habilita la auditoría y haz una auditoría sobre la tabla employees, al cambiar el sueldo de un empleado debe quedar registrado si aumenta más del 10%.
Haz unos update y mira la vista DBA_COMMON_AUDIT_TRAIL

Flashback

Permite volver a un estado anterior de los datos y por tanto recupera tablas o filas a un punto anterior en tiempo

Oracle Flashback utiliza el UNDO para obtener metadatos y datos históricos para las transacciones.

UNDO es persistente y sobrevive un cierre de base de datos y dura un tiempo pues proporciona consistencia de lectura para consultas de SQL.

Los datos de UNDO no duran siempre, son sobrescritos. Si intentamos acceder a versiones muy antiguas que ya han sido sobrescritas se produce un error de SNAPSHOT TOO OLD, habrá que aumentar el parámetro UNDO RETENTION lo que hará crecer el tablespace UNDO

Para habilitar el flashback tengo que tener la base de datos en modo recogida de datos suplementarios en el log:

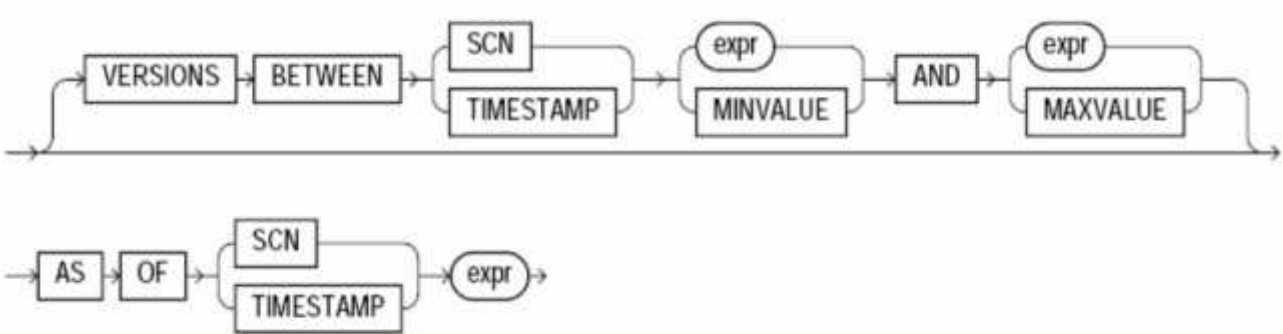
alter database add supplemental log data;

Existen dos estrategias distintas de flashback:

Flashback Version Query y Flashback Transaction Query

Flashback Version Query (FVQ)

Hace posible ver todos los cambios producidos en una tupla de la tabla en un período de tiempo concreto. FVQ permite añadir una cláusula **VERSIONS** a la sentencia SELECT que indique un **rango SCN o temporal** en el que se desea consultar las modificaciones. La sentencia devuelve una serie de metacampos con información sobre las versiones de la transacción.



METACAMPOS

- VERSIONS_STARTTIME. Data da primeira versión das tuplas devoltas pola consulta.
- VERSIONS_STARTSCN. Idem para SCN.
- VERSIONS_ENDTIME. Data da última versión das filas devoltas pola consulta.
- VERSIONS_ENDSCN. Idem para SCN
- VERSIONS_XID. Transacción que xerou a versión.
- VERSIONS_OPERATION. Operación que deu lugar á versión (I -insert-, U -update- o D -delete-).

Ejemplo, después de hacer dos updates:

```
SQL> select region_id,region_name,versions_xid,versions_startscn,versions_operation from  
HR.regions versions between SCN minvalue and maxvalue order by 1,versions_endscn desc;
```

REGION_ID	REGION_NAME	VERSIONS_XID	VERSIONS_STARTSCN	V
1	a Europa	06001D00FB250000	14381470	U
1	Europa	020003002F260000	14376223	U
1	Europe			
2	Americas			
3	Asia			
4	Middle East and Africa			

6 rows selected.

Puedo hacer ahora un select para que me dé los datos con la versión adecuada

```
select * from hr.regions as of scn 14376223;
```

Daría como resultado Europa

Podemos hacer lo mismo con el tiempo:

```
SQL> select * from hr.regions as of timestamp to_timestamp('10/01/15 18:45:00','DD/MM/YY  
HH24:MI:SS');
```

REGION_ID	REGION_NAME
1	Europe
2	Americas
3	Asia
4	Middle East and Africa

```
SQL> select * from hr.regions as of timestamp to_timestamp('19/01/15 18:45:00','DD/MM/YY  
HH24:MI:SS');
```

REGION_ID	REGION_NAME
1	Europa
2	Americas
3	Asia
4	Middle East and Africa

PRÁCTICA

FVQ

Crea una vista 'empleados_hace_una_hora' que muestre los datos de los empleados una hora antes de que se lance la query. Usa systimestamp-interval '60' MINUTE
Borra tres empleados al azar. Imagina que alguien los borró por error. Insértalos de nuevo con una sola sentencia. (ayuda: tienes que hacer un insert del select as of quitándole los que ya están para que no falle la PK)

Flashback Transaction Query (FTQ)

La vista flashback_transaction_query muestra información sobre las transacciones realizadas en la base de datos. De modificarse datos en una tabla, la vista muestra información sobre ellos y señala la sentencia precisa (en el campo UNDO_SQL) que permite deshacerlos.

Para usarla tenemos que garantizar el UNDO_RETENTION

alter tablespace <nombre_del_tablespace_undo> retention guarantee;

Entonces ya podemos usarla, por ejemplo, una vez seleccionada a xid :

```
SELECT xid, operation, start_scn, commit_scn, logon_user, undo_sql FROM flashback_transaction_query WHERE owner='HR';
```

```
SELECT xid, operation, start_scn, commit_scn, logon_user, undo_sql FROM flashback_transaction_query WHERE xid = HEXTORAW('008976780009872F');
```

Usamos la función HEXTORAW para que use el índice ya que el campo es de tipo RAW

También podemos hacer un join entre la tabla y la flashback_transaction_query.

PRÁCTICA

FTQ

Crea una vista para saber todos los cambios hechos en el esquema HR en un cierto período de tiempo (desde hace 1 hora).

PRÁCTICA

FTQ 2

Haz un par de updates en la tabla regions.
Selecciona los datos usando la vista anterior. Comprueba la información con FVQ
Deshaz la transacción realizada usando el undo_Sql